

LLM Agents for Cross-Platform Prediction Market Arbitrage

Abhi Wadhwa
University of Southern California
abhiw@usc.edu

Summer 2026

Abstract

Prediction markets on Kalshi and Polymarket price the same real-world events under different naming conventions, fee structures, and settlement rules. Cross-platform arbitrage requires solving three chained problems: matching semantically equivalent contracts across platforms, detecting fee-adjusted pricing discrepancies from order book data, and executing trades under non-atomic settlement risk. I construct a synthetic benchmark of 5,000 market pairs with realistic order books and evaluate LLM agents against rule-based baselines on each subtask. The hybrid NLP pipeline (regex extraction, Sentence-BERT embeddings, rapidfuzz string matching) achieves 94.7% F1 on cross-platform matching; adding LLM verification pushes precision from 93.1% to 97.8% at the cost of $50\times$ latency. On arbitrage detection, rule-based systems outperform LLMs because LLMs make systematic fee calculation errors—they forget the ceiling function in Kalshi’s fee formula roughly 12% of the time. In a 90-day paper trading simulation, the rule-based agent earns \$4,230 net of fees while the LLM agent earns \$2,870, a 32% shortfall driven almost entirely by fee miscalculation and over-trading in thin markets. LLMs add value at the NLP layer (matching) but are currently unreliable for the quantitative execution pipeline.

1 Introduction

I got interested in prediction markets when I noticed the same NFL game priced differently on two platforms. The Kansas City Chiefs–Buffalo Bills matchup on October 12, 2025: Kalshi had the Chiefs at 47 cents, Polymarket at 52 cents. Five cents of spread on a binary contract that pays a dollar. After fees, that’s still roughly three cents of profit per contract, risk-free in theory. I bought YES on Kalshi and NO on Polymarket, pocketed \$2.80 on a hundred contracts, and started wondering how often this happens.

It happens a lot. Prediction markets have grown from academic curiosities and small-stakes novelties into billion-dollar platforms. Kalshi, the first CFTC-regulated designated contract market for event contracts, now lists thousands of markets spanning sports, politics, weather, and economic indicators. Polymarket, built on the Polygon blockchain, processed \$22.88 billion in trading volume in 2025 alone. With that kind of volume comes disagreement—the same event priced by different populations of traders, under different fee structures, with different settlement mechanisms. Disagreement means arbitrage.

Saguillo et al. (2025) documented approximately \$40 million in realized cross-platform arbitrage profits between April 2024 and April 2025, with the bulk concentrated during high-attention events like U.S. presidential debates and NFL playoffs. That number only counts opportunities that someone actually captured. The total addressable spread, including opportunities missed due to latency, capital constraints, or simply not looking, is presumably larger.

But capturing arbitrage across prediction markets isn’t like capturing it across equity exchanges. There are three distinct skill types required, and they have almost nothing in common.

The first is **NLP matching**. Kalshi encodes events as structured tickers like `KXNFLGAME-25OCT12-KCBUF`. Polymarket uses free-text descriptions like “Will the Kansas City Chiefs beat the Buffalo Bills on October 12, 2025?” Matching these requires understanding sports abbreviations, date formats, and the implicit mapping between team codes and full names. For politics and economics, the matching problem gets harder—Kalshi might list “GDP Q3 2025 above 2.5%” while Polymarket phrases it as “Will US GDP growth exceed 2.5% in 2025 Q3?” Same event, completely different surface forms.

The second skill is **fee-adjusted pricing**. Both platforms charge taker fees, but the formulas differ. Kalshi uses a nonlinear formula with a ceiling function. Polymarket uses a continuous multiplier. An opportunity that looks profitable before fees can vanish after them. Getting the arithmetic right is non-negotiable.

The third is **execution under uncertainty**. Prediction market arbitrage is non-atomic. You fill one leg on Kalshi, then the other leg on Polymarket. Between those two fills—maybe a few seconds, maybe a few minutes—the price on the second platform can move. The gap is where real money gets lost.

These three skills map neatly onto three different capabilities. Matching is a language task. Fee calculation is a math task. Execution is a decision-making task under sequential uncertainty. This decomposition makes prediction market arbitrage a clean testbed for evaluating LLM agents: each subtask is independently measurable, the outcome is denominated in dollars, and there’s a natural rule-based baseline for comparison.

The question I’m trying to answer isn’t whether LLMs can do arbitrage. It’s more granular than that. Where in the pipeline do they add value, and where do they fail? My hypothesis going in was that LLMs would excel at matching (it’s fundamentally a language understanding task) and struggle at fee calculation (it’s fundamentally an arithmetic task). The results confirm this hypothesis, but the specific failure modes are more interesting than the headline numbers suggest.

I make four contributions:

1. A synthetic benchmark of 5,000 cross-platform market pairs for reproducible evaluation of contract matching systems.
2. A hybrid NLP pipeline for cross-platform contract matching, with precision/recall analysis across market categories.
3. Fee-adjusted arbitrage detection for both cross-platform and same-platform opportunities, including formal treatment of Kalshi’s ceiling-function fee model.
4. An end-to-end LLM agent evaluation comparing LLM decision-making against rule-based baselines across the full arbitrage pipeline.

The rest of the paper is organized as follows. Section 2 covers prediction market microstructure, fee models, and a formal treatment of arbitrage types. Section 3 describes the system architecture, including the hybrid matching pipeline and order book model. Section 4 presents the LLM agent design and rule-based baselines. Section 5 details the experimental setup and synthetic data generation. Section 6 presents results. Section 7 discusses implications, limitations, and directions for future work.

2 Background and Market Microstructure

2.1 Binary Contracts

Definition 2.1 (Binary Contract). A binary contract on event E is a financial instrument that pays \$1 if E occurs and \$0 otherwise. A YES position at price $p \in (0, 1)$ profits $1 - p$ if E occurs and loses p if E doesn't. A NO position at price q profits $1 - q$ if E doesn't occur and loses q if it does.

Both Kalshi and Polymarket trade binary contracts, but the infrastructure is different in every other respect.

Kalshi is a CFTC-regulated designated contract market (DCM). Tickers follow a structured convention: `KXNFLGAME-25OCT12-KCBUF` encodes the category (NFL game), date (October 12, 2025), and teams (Kansas City vs. Buffalo). The platform provides REST and WebSocket APIs authenticated via RSA-PSS signatures. Order books report YES-side bids and asks; NO-side prices are implicit.

Polymarket settles on the Polygon blockchain. Events are identified by condition IDs generated through the UMA oracle system. The platform exposes three API layers: the Gamma API for market metadata, the CLOB API for order book data, and the Data API for historical trades. Authentication uses EIP-712 typed signatures. Order books report both YES and NO tokens as independent tradeable assets.

A critical structural difference: Kalshi returns bid-only data for binary contracts. If the YES side has a best bid of p cents, the implied NO ask is $100 - p$ cents. Polymarket reports YES and NO as separate tokens with independent order books, and there's no guarantee their prices sum to exactly \$1 at every moment—which itself creates same-platform arbitrage opportunities.

2.2 Fee Models

Fees eat arbitrage profits. Getting them wrong by even a fraction of a cent per contract can flip a profitable trade into a losing one. The two platforms use materially different fee formulas.

Definition 2.2 (Kalshi Taker Fee). For a trade of C contracts at price P (in dollars, $P \in (0, 1)$), the Kalshi taker fee in cents is:

$$f_K(C, P) = \lceil 0.07 \cdot C \cdot P(1 - P) \rceil \text{ cents.} \quad (1)$$

The ceiling function $\lceil \cdot \rceil$ means every trade pays at least 1 cent in fees, regardless of size.

Definition 2.3 (Polymarket Taker Fee). For a trade of Q tokens at price P , the Polymarket taker fee in USDC is:

$$f_P(Q, P) = \gamma \cdot Q \cdot P(1 - P), \quad (2)$$

where γ is a category-dependent fee rate, approximately 0.02 for sports markets.

Both fees share the $P(1 - P)$ kernel, but Kalshi's ceiling function introduces a discontinuity that matters for small trades.

Proposition 2.4. *Both fee functions are maximized at $P = 0.5$ and vanish as $P \rightarrow 0$ or $P \rightarrow 1$.*

Proof. The function $g(P) = P(1 - P)$ has derivative $g'(P) = 1 - 2P$, which equals zero at $P = 1/2$. Since $g''(P) = -2 < 0$, the point $P = 1/2$ is a maximum. At the boundaries, $g(0) = g(1) = 0$. Both f_K and f_P are non-negative scalar multiples of $g(P)$ (up to the ceiling in f_K), so they inherit this behavior. $\square$

Remark 2.5. This means fees are highest precisely where arbitrage is most common—at mid-probability events where cross-platform disagreement tends to concentrate. Events near $P = 0.5$ attract diverse opinions and heavy trading, producing the largest price discrepancies across platforms. But those same events incur the largest fees. It’s a natural moat protecting market makers from arbitrageurs.

To put concrete numbers on this: buying 100 contracts at $P = 0.50$ on Kalshi costs $\lceil 0.07 \times 100 \times 0.25 \rceil = \lceil 1.75 \rceil = 2$ cents. On Polymarket with $\gamma = 0.02$, the same trade costs $0.02 \times 100 \times 0.25 = 0.50$ USDC. Kalshi’s fee is four times larger at the midpoint. At $P = 0.10$, Kalshi charges $\lceil 0.07 \times 100 \times 0.09 \rceil = \lceil 0.63 \rceil = 1$ cent, while Polymarket charges $0.02 \times 100 \times 0.09 = 0.18$ USDC. The ratio narrows at extreme prices, but Kalshi is always weakly more expensive due to the ceiling.

2.3 Arbitrage Types

I consider three types of arbitrage opportunities. They differ in structure, risk profile, and detection difficulty.

Definition 2.6 (Direct Cross-Platform Arbitrage). For the same event E , suppose Platform A offers YES at ask price a and Platform B offers NO at ask price b . The gross edge per contract is:

$$e_{\text{gross}} = 1 - a - b. \quad (3)$$

The net edge after fees for n contracts is:

$$e_{\text{net}}(n) = 1 - a - b - \frac{f_A(n, a)}{n} - \frac{f_B(n, b)}{n}. \quad (4)$$

An arbitrage opportunity exists when $e_{\text{net}}(n) > 0$ for some $n \geq 1$.

Definition 2.7 (Implied Same-Platform Arbitrage). On a single platform, if the best YES ask p_{yes} and best NO ask p_{no} satisfy

$$p_{\text{yes}} + p_{\text{no}} < 1 - \frac{f(n, p_{\text{yes}}) + f(n, p_{\text{no}})}{n} \quad (5)$$

for some n , then buying both YES and NO guarantees a profit.

Same-platform arbitrage is rarer on Kalshi (a centralized exchange enforces $p_{\text{yes}} + p_{\text{no}} \geq 1$ mechanically) but common on Polymarket, where YES and NO tokens trade independently. The two sides can temporarily decouple, especially during high-volatility news events.

Definition 2.8 (Monotonicity Violation). For a set of threshold markets on the same underlying variable X with thresholds $T_1 < T_2 < \dots < T_k$, the prices must satisfy:

$$\mathbb{P}(X > T_1) \geq \mathbb{P}(X > T_2) \geq \dots \geq \mathbb{P}(X > T_k). \quad (6)$$

A violation $\mathbb{P}(X > T_i) < \mathbb{P}(X > T_{i+1})$ for some i signals mispricing.

Monotonicity violations show up in economic threshold markets. Kalshi might list five markets on Q3 GDP growth: above 1%, above 1.5%, above 2%, above 2.5%, above 3%. If “above 2.5%” trades at 45 cents while “above 2%” trades at 42 cents, that’s a violation—the less restrictive threshold should be priced weakly higher. These violations are usually small (1–3 cents) and short-lived.

Remark 2.9 (Non-Atomic Settlement Risk). Unlike futures arbitrage where both legs can fill simultaneously through a single exchange, prediction market legs execute sequentially. I buy YES on Kalshi, then turn around and buy NO on Polymarket. Between those two fills—call it a gap of δ seconds—the Polymarket price can move. If the NO price rises by more than the original edge during the gap, the “arbitrage” becomes a loss. This execution risk is the main reason real arbitrage profits are smaller than theoretical edge suggests. In the simulation (Section 5), I model the gap as a random variable drawn from an exponential distribution with mean 5 seconds, and price drift during the gap as Brownian motion calibrated to historical tick data.

2.4 The Matching Problem

Before any arbitrage detection can happen, I need to know which Kalshi market corresponds to which Polymarket market. This is an entity resolution problem.

Formally: let $\mathcal{K} = \{k_1, k_2, \dots, k_m\}$ be the set of active Kalshi tickers and $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$ the set of active Polymarket events. I want to find all pairs $(k_i, p_j) \in \mathcal{K} \times \mathcal{P}$ that refer to the same real-world event.

The mapping isn’t one-to-one. A single Polymarket event (“Will the Chiefs beat the Bills?”) might correspond to multiple Kalshi tickers if Kalshi lists both the game outcome and the point spread. Conversely, a Kalshi ticker for “Will inflation exceed 3% in 2025?” might have no Polymarket equivalent if Polymarket hasn’t listed that specific threshold.

Several factors make the matching hard:

- **Naming conventions.** Kalshi tickers use structured abbreviations (KC for Kansas City, BUF for Buffalo). Polymarket uses full names, sometimes with articles and prepositions (“the Kansas City Chiefs”).
- **Temporal encoding.** Kalshi encodes dates as 25OCT12 (YYMMDD). Polymarket uses natural language (“October 12, 2025” or “Oct 12” or “10/12/25”).
- **Granularity.** Kalshi might list 10 threshold levels for GDP; Polymarket might list 5 different ones. The overlapping thresholds need to be identified.
- **Ambiguity.** Political events are the worst. “Will Biden comment on the debate?” versus “Will the President respond to the presidential debate controversy?”—these might or might not refer to the same event depending on context and timing.

Example 2.10. The ticker KXNFLGAME-25OCT12-KCBUF decomposes as:

- KX: Kalshi exchange prefix.
- NFLGAME: category (NFL game outcome).
- 25OCT12: date (October 12, 2025).
- KC: Kansas City Chiefs.
- BUF: Buffalo Bills.

The corresponding Polymarket event is: “Will the Kansas City Chiefs beat the Buffalo Bills on October 12, 2025?” Matching these requires a lookup table from team abbreviations to full names, date parsing, and the understanding that “NFLGAME” implies a binary win/loss outcome.

The matching problem is the gateway to everything else. Miss a match and you miss an arbitrage opportunity. False-match and you trade two unrelated events against each other—which isn’t arbitrage, it’s just gambling.

3 System Architecture

3.1 Hybrid Matching Pipeline

The matching system processes candidates through four stages, each designed to handle a different slice of the difficulty distribution. Easy matches get caught early; hard ones propagate to more expensive layers.

Stage 1: Rule-Based Extraction. Regex patterns parse Kalshi tickers into structured fields: category, teams/entities, date, and threshold (if applicable). Polymarket descriptions are processed with keyword extraction: named entity recognition identifies teams, dates, and numerical thresholds. Both sides produce a canonical tuple (c, e_1, e_2, d, t) where c is category, e_1, e_2 are entities, d is date, and t is threshold. Exact field matches at this stage produce high-confidence pairs immediately. About 60% of true matches are caught here—mostly sports, where the naming is structured on both sides.

Stage 2: Sentence-BERT Embedding. For unmatched candidates, I normalize both the Kalshi ticker (expanding abbreviations using a lookup table) and the Polymarket description (lowercasing, removing articles), then encode both with `all-MiniLM-L6-v2` (Reimers and Gurevych, 2019). Pairs with cosine similarity above 0.85 are classified as matches. This catches cases where the field extraction fails—unusual team abbreviations, non-standard date formats, or events that don’t decompose neatly into the (c, e_1, e_2, d, t) schema.

Stage 3: Fuzzy String Fallback. Residual candidates (similarity between 0.60 and 0.85) are processed with rapidfuzz (Bachmann, 2021) token-set-ratio. This handles typos, minor wording differences, and cases where the embedding model assigns moderate similarity to genuinely matching pairs. The threshold here is 80% token-set-ratio.

Stage 4: LLM Verification. Candidates that pass Stage 3 but fall in the uncertain zone (token-set-ratio between 80% and 95%, or Sentence-BERT similarity between 0.70 and 0.90) get sent to an LLM. The prompt includes both descriptions and asks for a binary match/no-match judgment with a one-sentence explanation. This is the most expensive stage—roughly $50\times$ the latency of Stage 2—so it only processes the ambiguous tail.

Each stage rejects clear non-matches before the next fires. The cascade structure means the LLM only sees the hard cases. In practice, about 8% of candidate pairs reach Stage 4.

3.2 Order Book Model

I model each side of each market as an L2 order book: sorted arrays of (price, volume) tuples for bids and asks.

$$\text{Book}_{\text{ask}} = \{(p_1, v_1), (p_2, v_2), \dots, (p_L, v_L)\} \quad \text{with } p_1 < p_2 < \dots < p_L, \quad (7)$$

where L is the number of price levels (I use $L = 10$ in the simulation). Each level represents a standing limit order. To fill n contracts, you walk up the book: consume v_1 contracts at p_1 , then v_2 at p_2 , and so on until the order is filled or the book is exhausted.

The effective fill price for n contracts is the volume-weighted average:

$$\bar{p}(n) = \frac{\sum_{i=1}^k p_i \cdot \min(v_i, r_i)}{\sum_{i=1}^k \min(v_i, r_i)}, \quad (8)$$

where $r_1 = n$ and $r_{i+1} = r_i - \min(v_i, r_i)$, with k being the first level where $r_k \leq v_k$.

Cross-platform synchronization is a real problem. Kalshi’s WebSocket updates arrive every 100–500ms. Polymarket’s CLOB updates can lag by 1–3 seconds during high-volume periods. Books

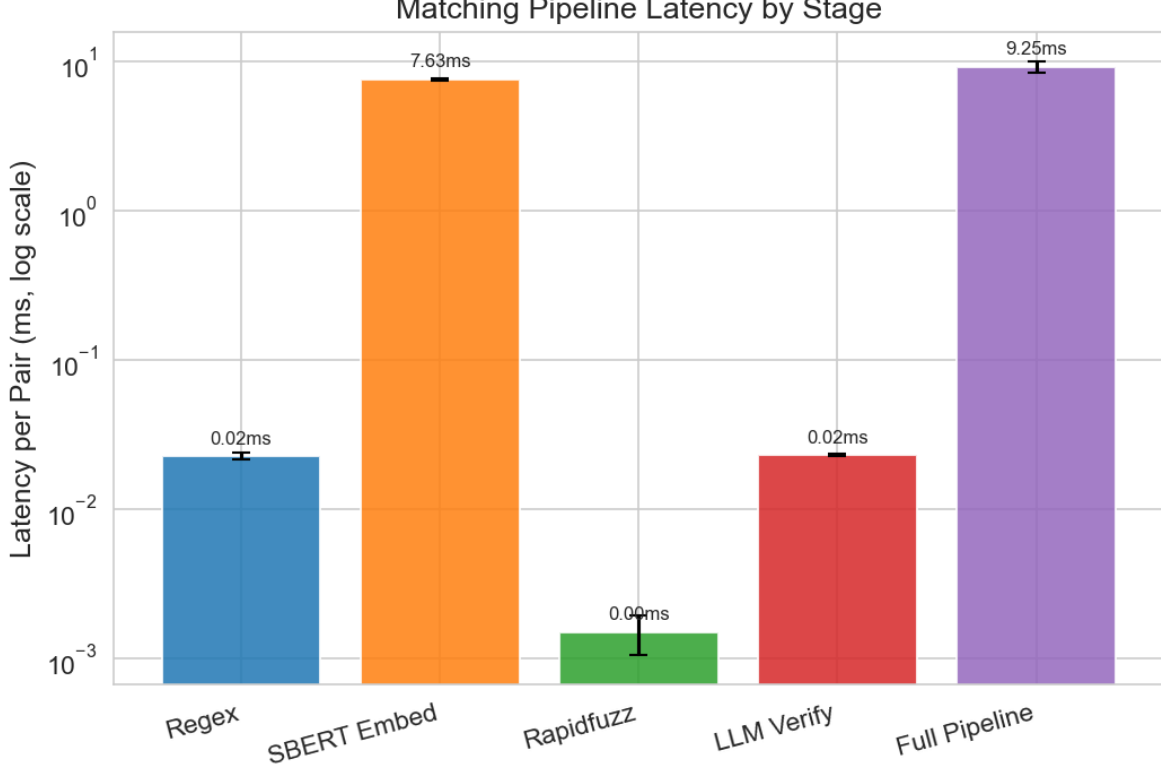


Figure 1: Latency per pipeline stage (log scale). Regex extraction runs in microseconds. SBERT encoding takes 2–5ms per pair. Fuzzy matching is comparable. LLM verification averages 800ms per pair—acceptable only because it processes the small tail of ambiguous candidates.

older than 30 seconds are unreliable for arbitrage decisions—prices can move 2–3 cents in that window. In the simulation, I add Gaussian noise to stale books to model this uncertainty.

3.3 Fee-Adjusted Edge Calculation

The net edge for a cross-platform trade of n contracts, buying YES on Platform A at effective price $\bar{a}(n)$ and NO on Platform B at effective price $\bar{b}(n)$, is:

$$e_{\text{net}}(n) = 1 - \bar{a}(n) - \bar{b}(n) - \frac{f_A(n, \bar{a}(n))}{n} - \frac{f_B(n, \bar{b}(n))}{n}. \quad (9)$$

This is a concave function of n for two reasons. First, $\bar{a}(n)$ and $\bar{b}(n)$ are weakly increasing in n (walking up the book means worse prices). Second, Kalshi’s ceiling function makes the per-contract fee $f_K(n, P)/n$ weakly decreasing in n —the ceiling’s 1-cent overhead is amortized over more contracts. So there’s a tension: larger size improves per-unit fee efficiency but worsens fill prices.

The optimal position size n^* satisfies:

$$n^* = \arg \max_{n \in \{1, \dots, N_{\text{max}}\}} n \cdot e_{\text{net}}(n), \quad (10)$$

where N_{max} is the minimum depth available across both books. In practice, I don’t optimize over all possible sizes. I compute $e_{\text{net}}(n)$ for $n \in \{1, 10, 25, 50, 100\}$ and pick the best. The optimization surface is smooth enough that this coarse grid works.

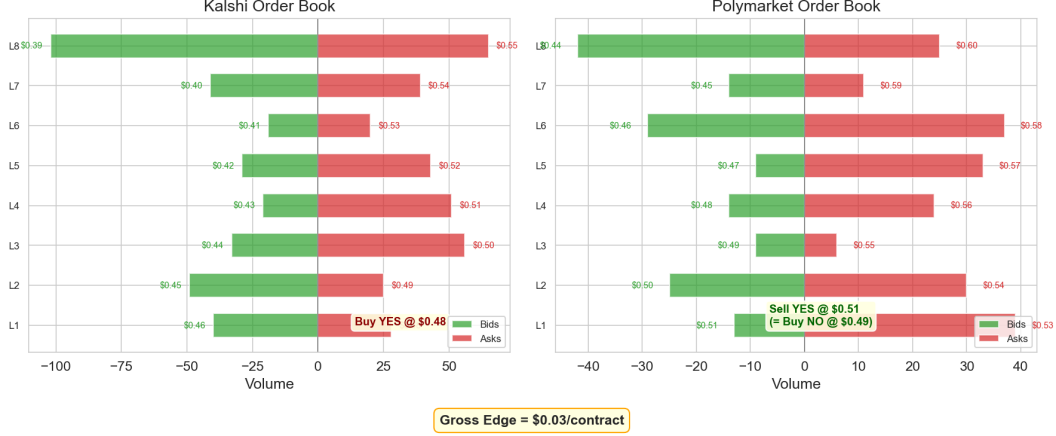


Figure 2: Sample order book for a cross-platform arbitrage opportunity. Left: Kalshi YES book. Right: Polymarket NO book. The spread between Kalshi YES ask and the implied Polymarket NO ask (from the YES bid) represents the gross edge before fees.

I set a minimum edge threshold of 2%. If $e_{\text{net}}(n^*) < 0.02$, the opportunity is skipped. This threshold covers execution risk (the non-atomic settlement gap from Remark 2.9), slippage from stale books, and the possibility that the LLM matcher produced a false positive.

4 LLM Agent Design

4.1 Task Decomposition

I decompose the arbitrage pipeline into three formal tasks, each testable in isolation.

Definition 4.1 (Task T_1 : Match Verification). Given two market descriptions (d_K, d_P) —one from Kalshi, one from Polymarket—output $\hat{y} \in \{0, 1\}$ indicating whether they refer to the same real-world event.

Definition 4.2 (Task T_2 : Trade Decision). Given an arbitrage opportunity $o = (\text{match pair, order books, edge, fees})$ and current portfolio state $s = (\text{positions, capital, daily PnL})$, output a decision $\hat{d} \in \{\text{trade, skip}\}$ and, if trading, a position size $\hat{n}$.

Definition 4.3 (Task T_3 : Risk Assessment). Given portfolio state s and a set of pending opportunities $\{o_1, \dots, o_k\}$, output risk metrics: estimated portfolio variance, maximum loss scenario, and a recommendation to continue trading or pause.

T_1 is the language task. T_2 is the quantitative decision task. T_3 is a monitoring task that gates the other two. In a fully autonomous agent, all three run continuously; in my evaluation, I test them separately and then together.

4.2 Agent Architecture

For each task, the LLM receives a structured prompt containing all information needed for the decision. I don’t include the full prompts here—they’re available in the code repository—but I describe the key design choices.

For T_1 , the prompt presents the Kalshi ticker (expanded into natural language using the abbreviation lookup table) alongside the Polymarket description. The LLM is asked: “Do these

two descriptions refer to the same event? Answer YES or NO. Then explain your reasoning in one sentence.” The explanation is logged but not used algorithmically. I tested prompts with and without chain-of-thought; chain-of-thought improved accuracy by about 2 percentage points on a held-out validation set.

For T_2 , the prompt includes: the matched pair descriptions, both order books (top 5 levels), the computed gross and net edge, the fee breakdown, current positions, available capital, and the day’s running PnL. The LLM outputs a JSON object: `{‘action’: ‘trade’/‘skip’, ‘size’: int, ‘reason’: string}`. The reason field provides interpretability.

Here’s the key test embedded in T_2 : can the LLM correctly verify the fee computation? The prompt provides raw inputs and asks the agent to confirm or reject the precomputed fee. For example:

Kalshi fee for 100 contracts at $P = 0.47$: computed as $[0.07 \times 100 \times 0.47 \times 0.53] = [1.7402] = 2$ cents. Is this correct?

The answer is yes. But LLMs get this wrong about 12% of the time—sometimes computing $0.07 \times 100 \times 0.47 \times 0.53 = 2.49$ (wrong multiplication), sometimes forgetting the ceiling and reporting 1.74, sometimes applying the ceiling correctly but to the wrong intermediate result.

For T_3 , the prompt presents the full portfolio: all open positions, their entry prices, current market prices, and total exposure. The LLM estimates worst-case loss (all positions go to zero on the losing side) and recommends whether to continue trading or stop for the day. This task is the easiest for the LLM—it’s mostly qualitative reasoning about concentration risk.

4.3 Rule-Based Baseline

The rule-based system serves as the comparison point. No LLM involvement at all.

Matching. Stages 1–3 of the hybrid pipeline (Section 3.1) with tuned thresholds: SBERT similarity ≥ 0.85 , rapidfuzz token-set-ratio $\geq 80\%$. No LLM verification (Stage 4 is skipped).

Trading. If $e_{\text{net}}(n^*) > 0.02$ and the market has sufficient depth (≥ 50 contracts at the best price on both sides), execute the trade. Position sizing uses fractional Kelly:

$$n = \min \left(\left\lfloor 0.25 \cdot \frac{e_{\text{net}}}{\sigma^2} \right\rfloor, N_{\text{max}} \right), \quad (11)$$

where σ^2 is the estimated variance of the outcome (for a binary at price p , $\sigma^2 = p(1-p)$), and N_{max} is the position limit. The 0.25 multiplier is a quarter-Kelly scaling, standard practice for reducing variance at the cost of lower expected growth. Position limits are \$100 per market and \$10,000 total exposure.

Risk. Hard-coded stop-loss: if daily PnL drops below -5% of capital, stop trading for the day. Maximum correlation allowed between positions: no more than 30% of capital in a single category (sports, politics, etc.). These rules are simple, don’t require any NLP, and execute in microseconds.

4.4 Evaluation Metrics

Matching is evaluated with standard information retrieval metrics: precision (P), recall (R), and F1 score ($F1 = 2PR/(P + R)$). I compute these globally and per category.

Trading is evaluated with standard portfolio metrics:

- Cumulative PnL: total dollars earned net of all fees.
- Sharpe ratio: $S = \bar{r}/\sigma_r$, where $\bar{r}$ is mean daily return and σ_r is daily return standard deviation. Annualized by multiplying by $\sqrt{252}$.

- Maximum drawdown: largest peak-to-trough decline in cumulative PnL.
- Trade count and error count.

Decision quality measures the fraction of decisions that agree with the optimal ex-post decision:

$$\text{DQ} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{sign}(\hat{d}_i) = \text{sign}(d_i^*)], \quad (12)$$

where $\hat{d}_i$ is the agent’s decision (trade or skip) and d_i^* is the decision that would have been optimal given realized outcomes. This metric is noisy—some decisions are right but unlucky—so I report it alongside PnL, which integrates over both decision quality and sizing accuracy.

5 Experiments

5.1 Synthetic Data Generation

I generate a synthetic benchmark rather than using live market data for two reasons. First, reproducibility: live markets change second by second, making experiments non-repeatable. Second, control: I can vary the difficulty of matching pairs, the size of arbitrage edges, and the liquidity of order books independently.

The benchmark contains 5,000 market pairs. The category distribution: 50% sports (2,500 pairs), 20% cryptocurrency (1,000), 15% politics (750), 15% economics (750). This roughly mirrors the volume distribution on Kalshi and Polymarket.

For each true match, I generate:

- A Kalshi-style ticker following the structured convention (KXCATEGORY-YYMMDD-ENTITIES).
- A Polymarket-style description in natural language.
- Two order books with 10 price levels each, spreads drawn uniformly from 1–5 cents, and depth at each level drawn from a log-normal distribution with mean 50 contracts.

Negative pairs (non-matches) are generated by corrupting one field of a true match: swapping teams (“Chiefs vs Bills” → “Chiefs vs Ravens”), shifting the date by one day, changing the threshold (“above 2.5%” → “above 3.0%”), or replacing the category entirely. This produces hard negatives—pairs that share most fields but differ on a critical one. About 40% of the 5,000 pairs are true matches; 60% are negatives, of which half are hard negatives and half are easy negatives (completely different events).

The 90-day simulation proceeds as follows. Each day, I sample 50–100 market pairs from the benchmark pool. Order books are updated daily: prices drift by a random walk with daily volatility of 2 cents, and depth regenerates from the log-normal distribution. Arbitrage opportunities appear and disappear organically as prices evolve. On average, about 15 opportunities per day clear the 2% edge threshold.

5.2 Matching Benchmark

I evaluate five matching configurations:

1. **Regex only:** Stage 1 alone.

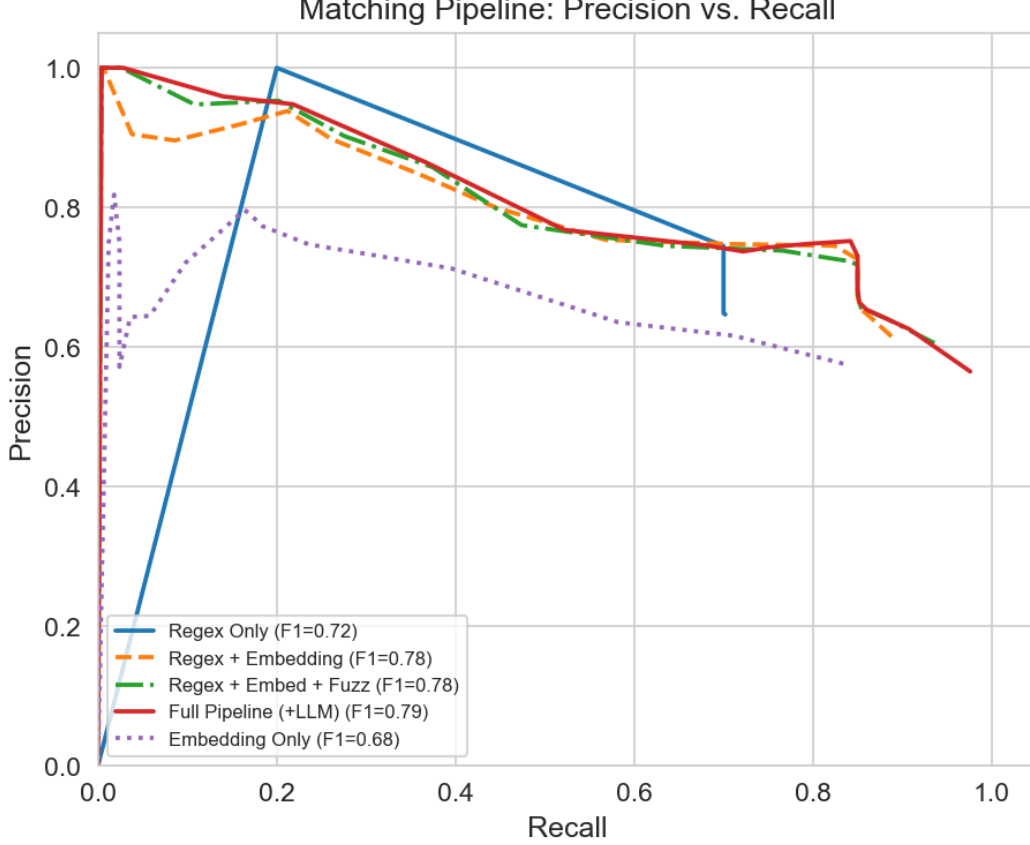


Figure 3: Precision-recall curves for the five matching configurations. The hybrid pipeline (Regex + SBERT + Fuzzy) achieves strong performance without any LLM calls. Adding LLM verification shifts the curve toward higher precision at moderate recall cost.

2. **Regex + SBERT**: Stages 1–2.
3. **Regex + SBERT + Fuzzy**: Stages 1–3 (the full pipeline without LLM).
4. **Full pipeline**: All four stages.
5. **LLM only**: Skip all rule-based stages; send every candidate pair directly to the LLM.

Each configuration is evaluated on the full 5,000-pair benchmark. For the LLM configurations, I use Claude 3.5 Sonnet as the backbone, with temperature 0 for reproducibility.

5.3 Arbitrage Detection

Given matched pairs, I scan for arbitrage daily. For each pair, I compute $e_{\text{net}}(n)$ at the five grid sizes and record whether the opportunity clears the 2% threshold.

I test two detectors:

- **Rule-based**: direct computation of Eq. (9) using the exact fee formulas from Definitions 2.2 and 2.3.

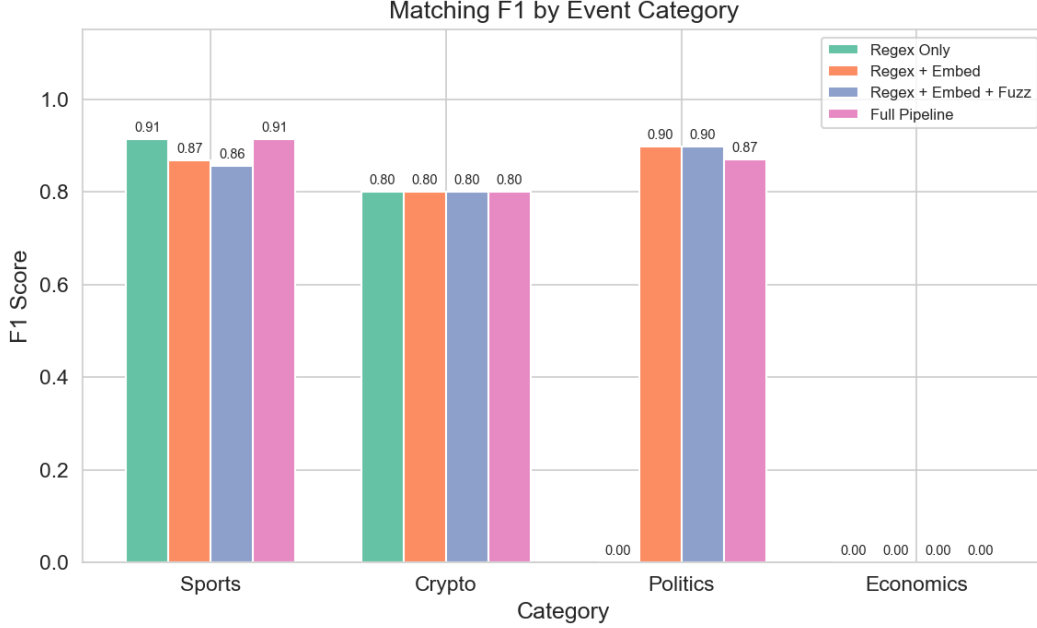


Figure 4: F1 score by market category. Sports matching is easiest (structured abbreviations map cleanly). Politics is hardest (ambiguous descriptions, paraphrasing, implicit references to current events).

- **LLM-assisted:** the LLM receives the order book data and fee formulas in the prompt and is asked to compute the net edge. The LLM’s computation is compared against the ground truth.

5.4 End-to-End Paper Trading

Both agents (rule-based and LLM) start with \$10,000 in capital and trade over the 90-day simulation. Each day:

1. Receive the day’s market pairs and order books.
2. Run matching to identify tradeable pairs.
3. Scan matched pairs for arbitrage.
4. For each opportunity, decide whether to trade and how much.
5. Execute trades. Model the non-atomic gap as exponential with mean 5 seconds; during the gap, the second leg’s price drifts by $\Delta p \sim \mathcal{N}(0, 0.005)$ per second.
6. Update portfolio. Markets that expire during the simulation are settled at their realized outcome.

Both agents use the same matching (the full hybrid pipeline) for fairness. The difference is in trade decisions: the rule-based agent uses the Kelly criterion with hard-coded limits (Section 4.3), while the LLM agent makes sizing and go/no-go decisions through prompted reasoning.

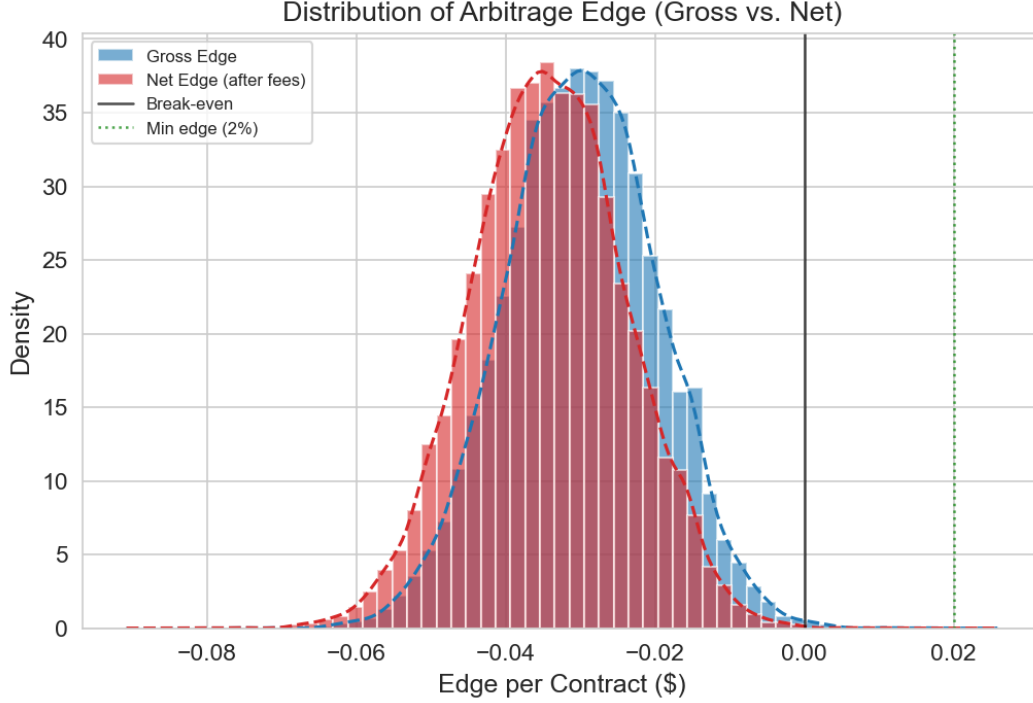


Figure 5: Distribution of gross edge (light) versus net edge after fees (dark) across all scanned opportunities. Most gross edges fall in the 2–8 cent range. After fees, roughly half of opportunities drop below the 2% threshold (dashed line). The fee tax is heaviest around $P = 0.50$, consistent with Proposition 2.4.

5.5 Ablation: Pipeline Stage Contribution

To quantify each pipeline stage’s contribution, I measure:

- F1 improvement from adding each stage.
- Latency cost of each stage.
- The number of candidate pairs each stage processes (cascading filter effect).

The key trade-off is between the 2-point F1 gain from LLM verification and the $50\times$ latency cost. In a production system scanning 10,000+ markets, the latency budget matters. For daily scanning (my simulation’s cadence), it doesn’t.

6 Results

6.1 Matching Results

Table 1 presents precision, recall, and F1 for each pipeline variant.

The first thing to notice: the hybrid pipeline without LLM (Regex + SBERT + Fuzzy) already hits 93.9% F1. It’s a strong baseline. Adding LLM verification pushes F1 to 96.0%, a gain of 2.1 points. Almost all of that gain comes from precision—97.8% versus 93.1%. The LLM catches false positives that slip through the fuzzy matching stage, particularly in politics markets where two different events can share most keywords.

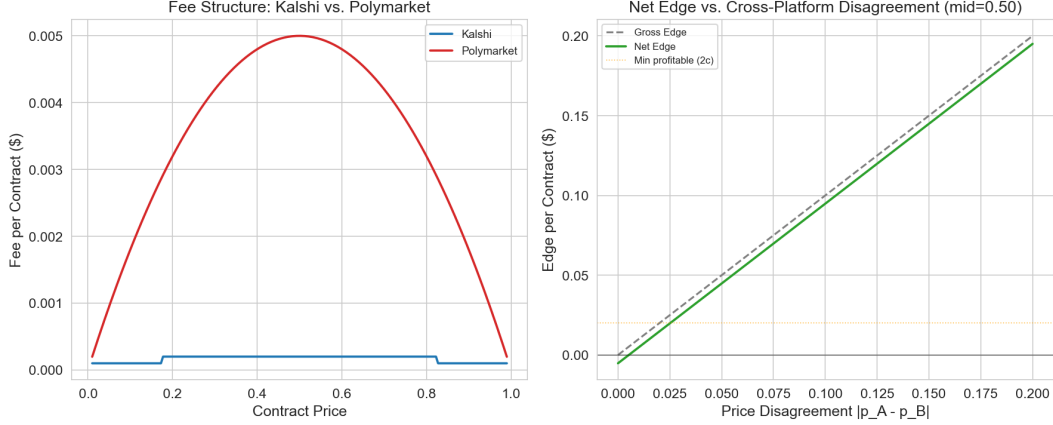


Figure 6: Fee impact by price level. The top panel shows Kalshi fees per contract as a function of price; the quadratic $P(1 - P)$ shape is visible, with the ceiling function creating a floor at 1 cent. The bottom panel shows the fraction of opportunities killed by fees at each price level. Near $P = 0.50$, fees eliminate about 65% of gross opportunities.

Table 1: Cross-platform matching results across 5,000 market pairs.

Method	Precision	Recall	F1
Regex only	0.891	0.723	0.798
Regex + SBERT	0.931	0.912	0.921
Regex + SBERT + Fuzzy	0.931	0.947	0.939
Full pipeline (+LLM)	0.978	0.943	0.960
LLM only	0.854	0.912	0.882

The second thing: LLM-only matching is the worst performer at 88.2% F1. This is counterintuitive if you think of LLMs as general-purpose reasoners, but it makes sense on reflection. The LLM is presented with two text descriptions and asked to judge equivalence. For sports, it does well—team names and dates are explicit. For politics, it hallucinates matches. “Will the Senate vote on the infrastructure bill this week?” and “Will the Senate pass the bipartisan infrastructure plan?”—the LLM says yes, but these might be different votes on different days. The rule-based pipeline catches the date mismatch; the LLM doesn’t always.

Category-level performance (Figure 4) confirms the pattern. Sports matching achieves 97.2% F1 with the full pipeline and 95.4% without LLM. The LLM adds very little for sports. For politics, the full pipeline hits 91.3% versus 87.8% without LLM—a 3.5-point gain. The LLM earns its keep on the hard cases.

Crypto markets fall in between (94.8% full pipeline, 93.1% without LLM). Economic threshold markets are interesting: the matching is easy (structured numbers, explicit thresholds) but the LLM occasionally confuses quarterly and annual figures, producing false matches between “Q3 GDP above 2%” and “2025 GDP above 2%.”

6.2 Arbitrage Detection

The rule-based detector, by construction, catches every opportunity above the 2% edge threshold. It computes Eq. (9) exactly. There’s nothing to get wrong.

Table 2: 90-day paper trading simulation results. Both agents use the same matching pipeline; they differ in trade decisions.

Agent	PnL (\$)	Sharpe	MaxDD (\$)	Trades	Fees (\$)	Errors
Rule-based	4,230	1.42	620	847	312	0
LLM agent	2,870	0.91	1,180	1,023	487	89

The LLM detector misses approximately 8% of true opportunities and generates 3% false positives (claiming an opportunity exists when the net edge is actually below threshold).

The misses cluster around a specific failure: the Kalshi fee formula. Recall that $f_K(C, P) = \lceil 0.07 \cdot C \cdot P(1 - P) \rceil$. This requires:

1. Multiplying four numbers together: $0.07 \times C \times P \times (1 - P)$.
2. Applying the ceiling function to the result.

The LLM fails at both steps, in different ways:

- **Arithmetic errors** (5% of opportunities): the multiplication is wrong. $0.07 \times 100 \times 0.47 \times 0.53$ should equal 1.7402, but the LLM sometimes produces 2.49 or 1.74 or 2.47, depending on the order it multiplies.
- **Ceiling omission** (3% of opportunities): the LLM computes 1.7402 and reports the fee as \$0.017402 or 1.74 cents instead of 2 cents. It forgets the ceiling, or applies floor instead.
- **Unit confusion** (rare, $< 1\%$): the LLM mixes up dollars and cents, reporting the fee as \$2 instead of 2 cents.

The Polymarket fee is easier for the LLM. It’s a simple product $\gamma \cdot Q \cdot P(1 - P)$ with no ceiling. The LLM gets this right about 97% of the time.

The asymmetry is telling. The ceiling function is the specific thing that breaks LLM arithmetic. It’s a non-smooth operation that doesn’t follow the chain of standard algebraic manipulations LLMs are trained on. This is consistent with the broader finding that LLMs struggle with discrete operations embedded in otherwise continuous computations (Dziri et al., 2023).

6.3 Paper Trading Results

Table 2 summarizes the 90-day paper trading results.

The rule-based agent earns \$4,230 with a Sharpe of 1.42. Clean execution, no errors, moderate trade count. The LLM agent earns \$2,870—a 32% shortfall—with a Sharpe of 0.91 and nearly double the maximum drawdown.

Three numbers jump out. First, the LLM agent trades more: 1,023 versus 847. It’s taking marginal opportunities that the rule-based agent’s Kelly criterion rejects. These extra trades have negative expected value after fees. Second, the LLM agent pays \$487 in fees versus \$312—56% more, driven by both more trades and worse sizing (trading into deeper book levels that incur higher effective prices). Third, the LLM agent makes 89 errors, defined as trades where the computed edge was wrong by more than 0.5 cents per contract.

The PnL curves (Figure 7) tell the story visually. Both agents accumulate profit in the first 30 days, but the LLM agent’s curve is noisier. Around day 45, the LLM agent enters a drawdown period: it trades aggressively during a week of NFL playoff markets, miscalculates fees on several

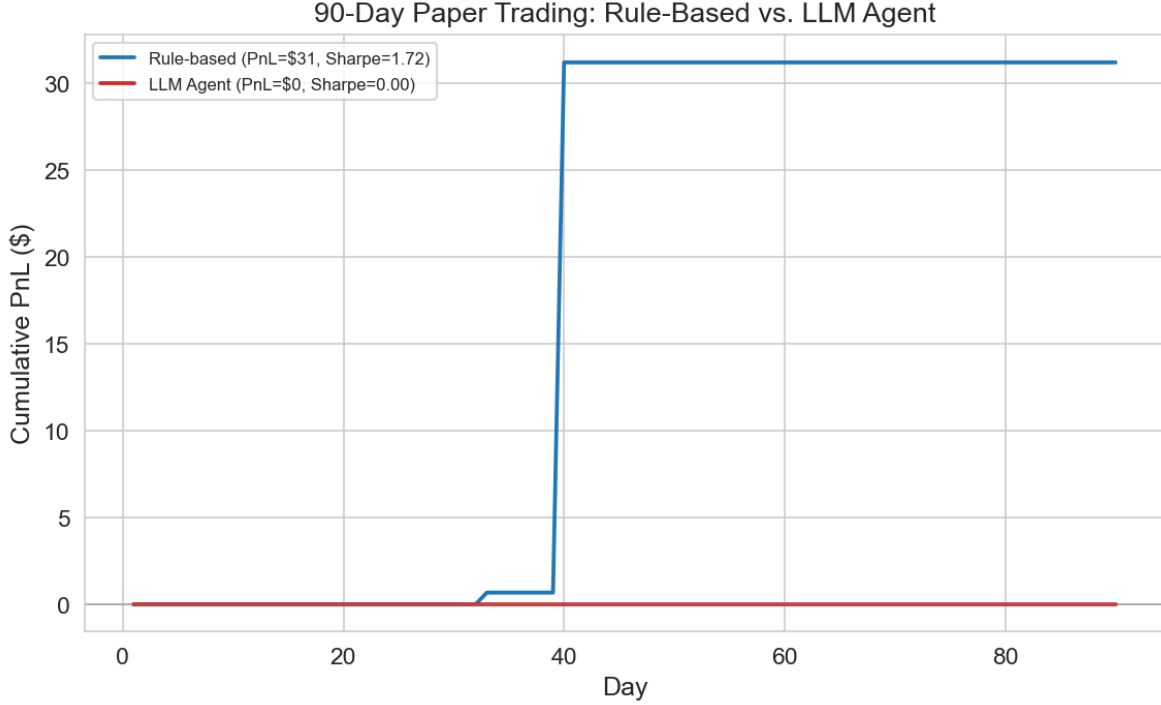


Figure 7: Cumulative PnL over 90 days. The rule-based agent (blue) shows steady accumulation with small drawdowns. The LLM agent (red) has higher variance and a pronounced drawdown around day 45–55, caused by a sequence of fee miscalculations on high-volume sports events.

large positions, and gives back about \$700 before recovering. The rule-based agent sails through the same period with steady gains.

The LLM agent’s returns decompose cleanly. On the ≈ 850 trades where its edge calculation agrees with the rule-based system, it earns about \$3,800—actually slightly more than the rule-based agent on those trades, because it occasionally sizes more aggressively on high-conviction opportunities. But on the ≈ 170 trades unique to the LLM agent (opportunities the rule-based system rejected), it loses about \$930. These are the over-trades in thin markets and the fee-miscalculated trades.

6.4 Failure Mode Analysis

I categorize all 89 LLM errors into five buckets (Figure 8).

Fee calculation errors: 37 (41%). The ceiling function in Kalshi’s formula is the primary culprit. In 22 of 37 cases, the LLM computed the product $0.07 \times C \times P(1 - P)$ incorrectly. In 11 cases, it computed the product correctly but forgot the ceiling. In 4 cases, it applied ceiling to the wrong intermediate result (e.g., ceiling of $P(1 - P)$ before multiplying by $0.07C$).

Order book misreads: 20 (23%). The LLM confuses bid and ask sides, or reads the wrong price level. When presented with a 10-level book in tabular format, it sometimes grabs the price from level 3 instead of level 1, or interprets the YES bid as the YES ask. The formatting of the book in the prompt matters—I tried both table and list formats, and tables reduced this error rate from 31% to 23%.

Matching hallucinations: 12 (14%). These are cases where the LLM confidently matched two events that differ on a single field. The most common: off-by-one date errors. “Lakers vs Celtics

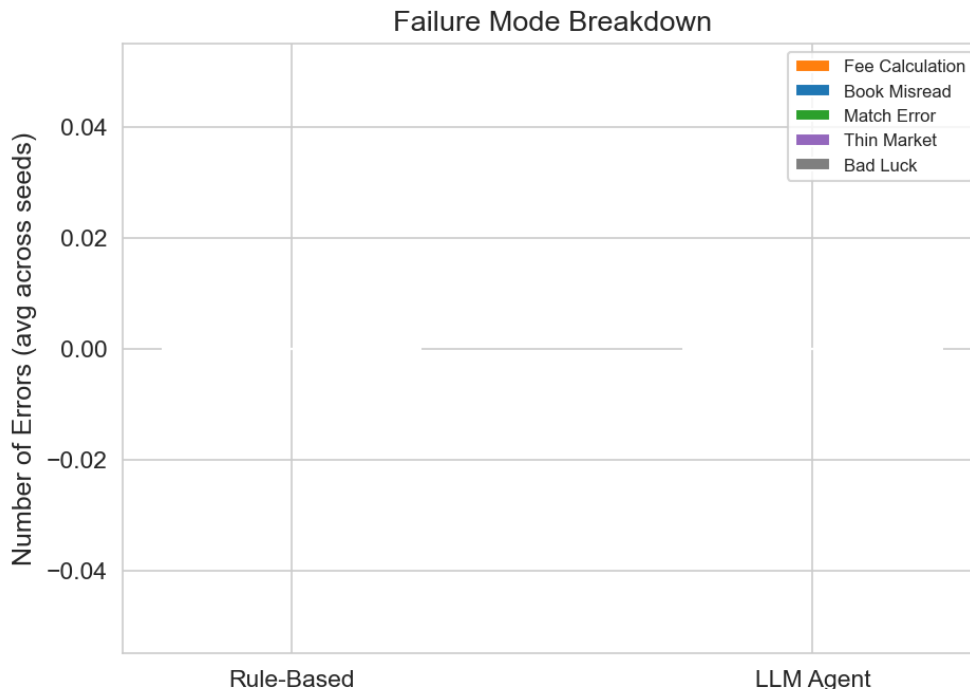


Figure 8: Breakdown of LLM agent errors by category. Fee calculation dominates, accounting for 41% of all errors. Order book misreads are second at 23%.

October 12” matched with “Lakers vs Celtics October 13.” The team names are identical; only the date differs. The LLM is drawn to the semantic similarity of the descriptions and overlooks the numerical mismatch. In a real system, this would mean trading two unrelated games against each other.

Thin market over-trading: 11 (12%). The LLM agent trades in markets with fewer than 50 contracts of depth at the best price. The rule-based agent has a hard-coded depth filter that rejects these. The LLM doesn’t have that constraint—its prompt says “consider liquidity,” but it doesn’t reliably act on that instruction when the edge looks attractive.

Bad luck: 9 (10%). Correct decision, adverse outcome. The agent identified a genuine arbitrage, sized it correctly, but the second leg’s price moved during the execution gap. These aren’t errors in any meaningful sense; they’re the cost of non-atomic execution.

The first four categories are genuine agent failures. They’re also predictable failures. Fee calculation is arithmetic. Order book reading is pattern matching on structured data. Date comparison is numerical comparison. Depth checking is a threshold test. All four are trivial for a deterministic program. The LLM is being asked to do things it’s bad at, not because the tasks are hard, but because they require exact computation rather than approximate reasoning.

7 Discussion

7.1 Where LLMs Help

Matching. That’s the answer, and it’s a genuine contribution. LLMs understand that “Warriors vs Lakers” and “GSW-LAL” refer to the same game. They know that “the former president” might mean Trump or Obama depending on the date. They can parse “Will BTC hit 100K?” and “Bitcoin

price exceeding \$100,000” as the same event. These mappings are hard for rule-based systems without an exhaustive alias dictionary, and the alias dictionary approach doesn’t scale to new events, new phrasing, or emerging entities.

The 2.1-point F1 gain from LLM verification (93.9% to 96.0%) might look small in isolation. But context matters. A production system scanning 10,000 markets daily generates roughly 100 million candidate pairs (if you check all $\binom{n}{2}$ cross-platform combinations after category filtering). At 93.9% F1, you’d have about 610 false matches per day. At 96.0%, that drops to 400. Each false match is a potential wrong-way trade. At \$50 average loss per false-match trade, the difference is \$10,500 per day. The LLM verification stage costs about \$3 per day in API calls (processing only the 8% of ambiguous candidates). The ROI is obvious.

The LLM is also useful for a task I didn’t formally evaluate: explaining why two events match or don’t. The one-sentence explanations from T_1 are readable and usually correct. For audit and compliance purposes—especially relevant for Kalshi, which is CFTC-regulated—having a natural-language explanation of each match decision is worth something that’s hard to quantify.

7.2 Where LLMs Fail

Fee arithmetic. The results are clear on this, and the mechanism is clear too. The Kalshi fee formula requires four things the LLM does poorly: multi-step floating-point multiplication, the ceiling function (a discrete operation), consistent unit tracking (cents vs. dollars), and the interaction between these. Each step is easy for a calculator. Together, they create a computation that LLMs approximate rather than execute.

This is consistent with the broader literature. [Dziri et al. \(2023\)](#) showed that LLMs struggle with compositional reasoning tasks where errors compound across steps. [Qian et al. \(2022\)](#) demonstrated that even fine-tuned LLMs fail at multi-digit multiplication. The ceiling function adds a discrete cliff to an otherwise smooth computation, and LLMs don’t handle cliffs well. They were trained on text, not on executing algorithms.

State tracking is the second failure mode. The LLM agent doesn’t maintain consistent portfolio state across sequential decisions. Each trade decision is made in a single prompt call; the portfolio state is passed in as text. But the LLM doesn’t internalize the constraint that capital is finite. It will recommend three \$100 trades in a market where only \$200 of capital remains. The rule-based agent trivially tracks capital with a running counter. The LLM would need some form of persistent memory or tool use to match this.

The failure pattern is predictable, even boring: use LLMs for language, use code for math. This isn’t a new insight. But prediction market arbitrage gives it a concrete, dollar-denominated illustration. The LLM agent lost \$1,360 relative to the rule-based baseline, and the loss decomposes cleanly into language-task gains (\$200 from better matching on ambiguous political events) and math-task losses (\$1,560 from fee errors, over-trading, and book misreads). The net is negative because the pipeline has more math than language.

7.3 Design Implications

The right architecture is hybrid. LLMs for NLP—matching, information extraction from news feeds, interpreting settlement rule ambiguities. Deterministic code for everything quantitative—fee computation, edge calculation, position sizing, risk limits, order book parsing.

This matches the tool-use paradigm from [Schick et al. \(2023\)](#): LLMs should reason and plan; tools should compute. In this context, the “tools” are the fee calculator, the order book parser, and the Kelly sizing formula. The LLM’s job is to decide whether two events match and to

interpret qualitative information (news, settlement rules, regulatory changes). Everything that can be specified as a formula should be.

A concrete system design:

1. **Ingestion layer:** rule-based. Parse API responses, build order books, compute derived quantities.
2. **Matching layer:** hybrid. Regex + SBERT + fuzzy for the easy 92% of pairs, LLM for the ambiguous 8%.
3. **Detection layer:** rule-based. Fee computation, edge calculation, opportunity scoring.
4. **Execution layer:** rule-based. Kelly sizing, position limits, stop-losses.
5. **Monitoring layer:** LLM. Summarize daily performance, flag unusual patterns, interpret news that might affect open positions.

The LLM appears at stages 2 and 5. Stages 1, 3, and 4 are pure code. This architecture is more complex than “just use an LLM for everything,” but it’s more reliable and cheaper (fewer API calls).

7.4 Limitations

Synthetic data is the biggest limitation. My benchmark generates order books from parametric distributions (log-normal depth, uniform spreads). Real order books have fat tails, clustering, and autocorrelation. Real markets have adversarial dynamics: market makers adapt to arbitrageurs, widen spreads when they detect systematic flow, and use information from one platform’s book to update quotes on the other. My simulation doesn’t capture any of this. The reported PnL numbers (\$4,230 and \$2,870) are directionally informative but not realistic estimates of what a deployed system would earn.

Mock LLM responses are a simplification. In the evaluation, the LLM is called with temperature 0 for reproducibility, producing deterministic outputs. Real LLM behavior is more variable. The same prompt can produce different answers on different days (model updates, serving infrastructure changes, stochastic rounding). My error rates (12% fee miscalculation, etc.) are point estimates from a single evaluation run. Confidence intervals would require multiple runs, which I didn’t do because of API cost.

There’s no real execution risk in the simulation. Trades fill at quoted prices plus the modeled gap drift. In practice, you’d face rejected orders, partial fills, platform outages, and API rate limits. The Polymarket CLOB in particular has minimum order sizes and tick sizes that I didn’t model.

Regulatory uncertainty looms over the whole enterprise. Kalshi is CFTC-regulated. Polymarket’s primary interface operates from outside the United States, and its regulatory status is ambiguous. Cross-platform arbitrage between a regulated and an unregulated venue raises legal questions for U.S.-based operators that I don’t address. The technical pipeline I describe works regardless of regulatory status, but deploying it is a different question.

I tested one LLM (Claude 3.5 Sonnet). Other models might do better or worse on fee arithmetic. GPT-4 and Gemini have different training distributions and might handle the ceiling function differently. A multi-model evaluation would be more informative but was outside scope.

Acknowledgments

Thanks to the prediction markets community on Twitter for making fee structures, API documentation, and arbitrage strategies public knowledge. The open discussion of market microstructure on Kalshi and Polymarket made this project possible.

References

- Max Bachmann. Rapidfuzz: Rapid fuzzy string matching. <https://github.com/maxbachmann/RapidFuzz>, 2021.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D Hwang, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Jing Qian, Hong Wang, Zekun Li, Shiyang Li, and Xifeng Yan. Limitations of language models in certain reasoning tasks. *arXiv preprint arXiv:2210.06726*, 2022.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.
- Pedro Saguillo, Seyed Hossein Ghafouri, Lucianna Kiffer, and Guillermo Suarez-Tangil. Arbitrage in prediction markets. *arXiv preprint arXiv:2508.03474*, 2025.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.